MCP with AI Agents

Breakout: Build and Secure a Custom MCP Server

Day 1 - 45 minutes

KavinSchool

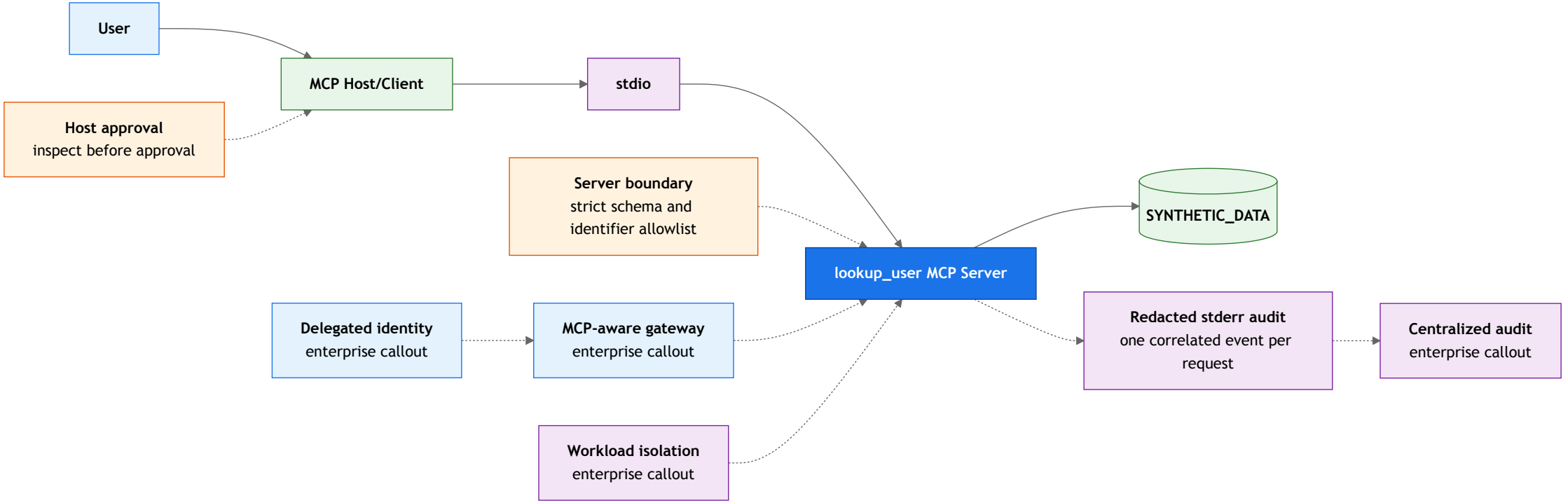
Mission and Pair Roles

Complete a bounded, read-only `lookup_user` tool over `stdio` and prove that its server-side controls deny prohibited requests.

- **Output:** passing verifier, redacted audit evidence, and annotated architecture map
- **Driver:** types, runs commands, and applies reviewed changes
- **Navigator:** reads the contract, predicts results, and checks security boundaries
- **Switch:** exchange roles after Phase 2
- **Before first tests:** predict pass or fail privately, then compare before viewing output

Use one selected client: Claude Code or GitHub Copilot in VS Code.

Request Flow and Security Boundaries



Text alternative: User to MCP Host/Client to stdio to `lookup_user` MCP Server to `SYNTHETIC_DATA`. Approval belongs at the host. Strict schema and identifier allowlist belong at the server. Redacted audit leaves through stderr. Enterprise additions are delegated identity, an MCP-aware gateway, workload isolation, and centralized audit.

Open the Lab and Establish Baselines

From the companion repository:

```
cd day1/breakout-build-secure-server
pwd
node --version
npm --prefix ../prebuilt-readonly-server ci
npm --prefix ../prebuilt-readonly-server run verify
```

- Path must end in `day1/breakout-build-secure-server`; Node must be 22 or later.
- Learner edits belong only in `start/server.ts`.
- Do not edit verifiers, package files, lockfiles, or `tsconfig.json`.

Run of Show: 45 Minutes

Time	Phase	Evidence
5 min	Connect the pre-built server	One read tool, synthetic result, denied path, approval observation
15 min	Complete the handler	Intended red state and reviewed bounded TODO patch
8 min	Add the security boundary	Strict validation, allowlist, and redacted audit
10 min	Verify, register, and repair	Four PASS lines or selected-client evidence
7 min	Review, map, and share out	Annotated map and 60-second explanation

The README contains the full prompts, client registration steps, and troubleshooting.

Checkpoint 1: Connect and Complete

Phase 1, 5 min: verify the pre-built server, choose one client, inspect its trust prompt, and record the approval observation.

```
PREBUILT_ROOT="$(cd ../prebuilt-readonly-server && pwd)"
printf '%s\n' "$PREBUILT_ROOT"
```

Phase 2, 15 min: establish the intended red state, then complete only the bounded handler TODOs described in the README.

```
npm --prefix start ci
npm --prefix start run build
npm --prefix start run verify
```

Review the diff together. Reject changes outside `start/server.ts`, weakened tests, argument logging, broad `any` types, or new capabilities.

Demo: Connect Pre-built Server to Claude

From the lab root after the verifier passes:

```
PREBUILT_ROOT="$(cd ../prebuilt-readonly-server && pwd)"
claude mcp add --scope local --transport stdio course-readonly-users \
  -- node "$PREBUILT_ROOT/server.mjs"
claude mcp get course-readonly-users
```

Start `claude` from the lab root, enter `/mcp`, and confirm that only `read_course_users` is available.

Demo: Ask Claude About Course Users

```
Use read_course_users to identify every active user and report each
user's role. Show the proposed tool call before using it.
```

- **Tool:** `read_course_users`
- **Arguments:** `{}`
- **Answer:** `user_1` is active with the `course-admin` role.
- **Boundary:** No path argument or mutation tool is available.

> Keep this server connected until the cleanup step in the README.

Checkpoint 2: Add the Security Boundary

Phase 3, 8 min: switch roles and implement the remaining allowlist and audit behavior.

- Preserve strict parsing and extra-property rejection.
- Allow only identifiers present in `SYNTHETIC_DATA`.
- Return generic `isError` denials.
- Audit every request with timestamp, fresh correlation ID, tool, decision, and outcome.
- Never audit arguments, identifiers, validation details, or returned data.

```
npm --prefix start run build
npm --prefix start run verify
```

Use the full bounded coding-assistant prompts in the README. Explain and review each patch before applying it.

Checkpoint 3: Verify, Register, and Repair

Phase 4, 10 min: run the exact acceptance command.

```
npm --prefix start run verify
```

Expected green evidence:

```
PASS discovered only lookup_user
PASS user_1 returned the expected synthetic record
PASS denied admin_999, malformed syntax, extra properties, and unknown tool
PASS errors and five audit events are redacted
```

Register and use the built custom server in the same client. If host integration fails, keep the passing verifier as equivalent fallback evidence. Use exact failure output with the README repair prompt.

Use the Server: Connect with Claude Code

From the lab root after the build succeeds:

```
LAB_ROOT="$(pwd)"
claude mcp add --scope local --transport stdio course-secure-lookup \
  -- node "$LAB_ROOT/start/dist/server.js"
claude mcp get course-secure-lookup
```

- Start `claude` from this lab root.
- Enter `/mcp` and confirm that only `lookup_user` is available.

> VS Code users follow the matching `.vscode/mcp.json` instructions in the README.

Use the Server: Prove the Allow Path

Ask Claude:

```
Use lookup_user to look up user_1. Show the tool name and arguments
for approval, then summarize the result.
```

- **Tool:** `lookup_user` with `{"id":"user_1"}`
- **Result:** `Role: Admin, Status: Active`
- **Audit:** `decision=ALLOW` and `outcome=success`

> The audit event must not contain the identifier, arguments, or returned record.

Use the Server: Prove the Deny Path

Ask Claude:

```
Use lookup_user to look up admin_999. Show the tool name and
arguments for approval, then report the server response.
```

- **Host:** Approval permits the proposed call to reach the server.
- **Server:** The identifier allowlist rejects `admin_999` with `isError`.
- **Audit:** `decision=DENY` and `outcome=id_not_allowed`

> Host approval does not override validation or authorization inside the server.

Architecture Map and 60-Second Share-Out

Phase 5, 7 min: draw and label this exact path:

```
User -> MCP Host/Client -> stdio -> lookup_user MCP Server -> SYNTHETIC_DATA
```

Annotate host approval, strict schema and identifier allowlist, and redacted stderr audit. Add enterprise callouts for delegated identity, MCP-aware gateway, workload isolation, and centralized audit.

Each pair shows:

- Four PASS lines or equivalent client evidence
- One redacted audit structure and the annotated map
- One private prediction that changed after evidence
- Why server-side control still denies `admin_999` after host approval

Definition of Done and Help Order

Done: one discovered tool; exact `user_1` synthetic result; `admin_999`, malformed syntax, extra properties, and unknown tool denied with generic `isError`; all five requests audited without sensitive data; client or fallback evidence captured.

Help order:

1. Ask your partner and re-read the relevant README phase.
2. Compare `start/server.ts` with `start/verify.mjs` and preserve verifier checks.
3. Run `npm --prefix start run verify` and keep the exact failure output.
4. Use the bounded README repair prompt and review the smallest patch together.

If host setup blocks progress for two minutes, use the documented verifier fallback. Stretch work is the README threat note only; do not add code or capabilities.

Return for Kahoot Quiz 2

Bring your PASS evidence, redacted audit structure, and annotated architecture map.

KavinSchool

Let's Connect

LinkedIn: linkedin.com/in/kpassoubady



Kangeyan Passoubady

Principal Automation Architect